reading_data_formats

June 16, 2026

1 Reading Data: URLs, File Formats, and Data Frame Types

This is a short companion to Lecture 5 (Introduction to Datasets). We will cover three practical skills:

1. **Reading data from a URL** — how to load a CSV directly from a GitHub raw link, no downloading required.
2. **Working with Parquet files** — a columnar file format that is much more efficient than CSV for large datasets.
3. **Converting between data frame types** — `data.frame` (base R), `tibble` (tidyverse), and `data.table` (fast, big-data-friendly).

We will use a global CO2 emissions dataset maintained by Our World in Data throughout.

[ ]:
```r
# ------------------------------------------------------------------------
# Housekeeping: install packages if needed, then load
# ------------------------------------------------------------------------
if (!require(dplyr))      install.packages("dplyr")
if (!require(readr))      install.packages("readr")
if (!require(ggplot2))    install.packages("ggplot2")
if (!require(arrow))      install.packages("arrow")
if (!require(data.table)) install.packages("data.table")
if (!require(tidyr)) install.packages("tidyr")

library(dplyr)
library(readr)
library(ggplot2)
library(arrow)
library(data.table)
library(tidyr)
```

2 1. Reading Data from a URL

So far we have read CSVs stored on our own computer. But data is often hosted online — on government portals, institutional repositories, or GitHub. You can pass a URL directly to `read_csv()` just like a file path.

GitHub raw links are especially handy. When you are looking at a CSV on GitHub, click the **Raw** button to get the direct URL to the plain-text file. It looks like this:

`https://raw.githubusercontent.com/<user>/<repo>/<branch>/<path/to/file.csv>`

Copy that URL and hand it straight to `read_csv()`. No downloading, no saving to your hard drive.

[ ]:
```r
# ------------------------------------------------------------------------
# Read a CSV directly from a GitHub raw link
# Dataset: Global CO2 emissions - Our World in Data
# https://github.com/owid/co2-data
# ------------------------------------------------------------------------
url <- "https://raw.githubusercontent.com/owid/co2-data/master/owid-co2-data.
 ↪csv"

df <- read_csv(url)

head(df)
```

[ ]:
```r
# Quick look at the structure
glimpse(df)
```

3 Quick Plot: CO Emissions Over Time

Let's do a quick sanity check on our data with a visualization. We will filter to a handful of countries and plot annual CO emissions since 1950.

[ ]:
```r
df %>%
  filter(
    country %in% c("United States", "China", "Brazil", "India", "Germany"),
    year >= 1950
  ) %>%
  select(country, year, co2) %>%
  drop_na(co2) %>%
  ggplot(aes(x = year, y = co2, color = country)) +
  geom_line(linewidth = 1) +
  labs(
    title    = "Annual CO2 Emissions by Country",
    subtitle = "Source: Our World in Data",
    x        = "Year",
    y        = "CO2 emissions (million tonnes)",
    color    = "Country"
  ) +
  theme_minimal()
```

4 2. Parquet Files

A **Parquet** file is a columnar storage format — data is stored column-by-column rather than row-by-row. For large environmental datasets (think: satellite imagery metadata, national air quality monitoring, species occurrence records), Parquet has major advantages over CSV:

	CSV	Parquet
File size	Larger	Much smaller (compressed by default)
Read speed	Slower	Faster, especially when selecting specific columns
Data types	Guessed on load	Stored in the file — no surprises

The `arrow` package lets us read and write Parquet in R with almost no extra effort.

[ ]:
```r
# ------------------------------------------------------------------------
# Write the data as both CSV and Parquet, then compare file sizes
# ------------------------------------------------------------------------

write_csv(df,     "co2_data.csv")
write_parquet(df, "co2_data.parquet")

csv_kb     <- round(file.size("co2_data.csv")     / 1024, 1)
parquet_kb <- round(file.size("co2_data.parquet") / 1024, 1)

cat("CSV size:     ", csv_kb,     "KB\n")
cat("Parquet size: ", parquet_kb, "KB\n")
cat("Parquet is", round(csv_kb / parquet_kb, 1), "x smaller\n")
```

[ ]:
```r
# ------------------------------------------------------------------------
# Read a Parquet file back in - identical to reading a CSV
# ------------------------------------------------------------------------
df_from_parquet <- read_parquet("co2_data.parquet")

head(df_from_parquet)
```

5 3. Data Frame Types: `data.frame`, `tibble`, and `data.table`

In R, there are three common ways to represent tabular data. They do the same fundamental job but behave differently under the hood.

Type	Package	Best for
`data.frame`	base R	Simple scripts, no dependencies
`tibble`	tidyverse (`dplyr`, `readr`)	Most day-to-day work — clean printing, sensible defaults
`data.table`	`data.table`	Very large datasets where speed matters

When you use `read_csv()`, you already have a **tibble**. You can convert between types freely.

[ ]:
```r
# read_csv() returns a tibble - notice the three class labels
class(df)
```

When R reports the class of a tibble, you see three labels. Each one is a layer of what the object "is":

- `tbl_df` — marks it as a tibble specifically. This is what triggers the nice printing behavior (only the first 10 rows, column types shown below the names, no row numbers cluttering the output).
- `tbl` — a more general "table" class, used internally by the tidyverse as a shared interface.
- `data.frame` — the base R data frame. Every tibble is also a valid `data.frame`, which is why all standard base R functions work on tibbles without any changes.

The three classes stack from most-specific to least-specific. R reads them left to right and uses the first one it knows how to handle — so tidyverse functions see a tibble, and base R functions see a plain data frame.

[ ]:
```r
# Convert to a base R data.frame
df_base <- as.data.frame(df)
class(df_base)
```

[ ]:
```r
# Convert to a data.table
df_dt <- as.data.table(df)
class(df_dt)
```

5.0.1 `data.table` — pros and cons

Pros: - **Speed.** `data.table` is one of the fastest data manipulation tools in R. On datasets with millions of rows it can be 10–100x faster than base R and noticeably faster than `dplyr`. - **Memory efficiency.** It modifies data in place (no copy made), which matters when RAM is tight — relevant for large environmental monitoring or remote sensing datasets. - **Concise syntax.** Its `DT[i, j, by]` syntax (filter rows, select/compute columns, group) can do a lot in one line.

Cons: - **Steeper learning curve.** The `DT[i, j, by]` syntax is compact but unfamiliar if you are coming from the tidyverse. It has its own conventions that take time to get comfortable with. - **Less readable.** For people new to R, `data.table` code is harder to parse at a glance compared to the plain-English feel of `dplyr` verbs. - **Overkill for most work.** Unless your dataset has millions of rows or speed is a real bottleneck, a tibble + `dplyr` is simpler and more than fast enough.

Bottom line: Use a tibble for almost everything. Reach for `data.table` when you hit a real performance wall.

[ ]:
```r
# Convert back to a tibble from any of the above
df_tibble <- as_tibble(df_dt)
class(df_tibble)
```